

Aula 04

Mineração de Dados



**Departamento de
Computação**



LEITURA E ESCRITA DE DADOS

Objetivos

- ***Leitura e Escrita de Dados***

- *Apresentar as diversas funções que permitem a leitura de dados em diversos formatos*
- *Demonstrar como carregar os objetos em R*
- *Detalhar como escrever/transferir dados em R para arquivos*



- ***Definição do Diretório de Trabalho***

- *Em cada sessão do R, existe sempre um **diretório** (ou pasta) **definido** como o **diretório atual** de **trabalho***
- *Será neste **diretório** que o R, por padrão, **procurará arquivos** para **carregar** e **gravar** os **arquivos***
- *Em cada função de **leitura** ou **escrita**, pode ser **definido** a **localização** do **seu arquivo** de **duas formas**:*
 - ***Relativa**: definindo como base o diretório de trabalho*
 - ***Absoluta**: definindo o caminho completo desde a raiz*

- ***Definição do Diretório de Trabalho***

- **Exemplo (01):**

- ***Obtendo o diretório atual de trabalho*** por meio da **função** `getwd`
 - ***É permitido alterar o diretório atual de trabalho*** utilizando a **função** `setwd`

```
> getwd()
```

```
[1] "C:/Users/Geraldo/Documents"
```

```
> setwd("C:/temp/datascience")
```

- **Definição do Diretório de Trabalho**

- No *Microsoft Windows* os caminhos podem usar o símbolo **** no lugar de **/**
- Sendo algo do tipo “*C/user/documents*” no *Windows* ou “*/Users/user1*” em *Mac OS X* ou *Linux*



Mac OS X



- ***Definição do Diretório de Trabalho***

- **Exemplo (02):**

- *Configurando o **diretório atual de trabalho** utilizando o **caminho relativo***

```
> setwd("/temp/datascience")
```

```
> getwd()
```

```
[1] "C:/temp/datascience"
```

- ***Definição do Diretório de Trabalho***

- ***Observações:***

- Na ***instalação*** do R ***existem opções*** nos ***menus*** da ***interface gráfica*** que ***permitem mudar*** o ***diretório de trabalho***
 - Os IDE ***permitem realizar*** esta ***operação*** com ***interfaces próprias***
 - No RStudio, o ***diretório de trabalho*** pode ser ***definido*** no ***canto inferior direito***, no ***Tab Files***, que ***permite navegar*** no ***sistema de arquivos*** e ***definir*** a ***pasta selecionada*** como ***diretório de trabalho***

- **Leitura de Dados**

- A função **scan**, permite ler valores diretamente por meio da **linha de comandos**
- Dessa forma, o **conjunto de valores introduzidos** são lidos **como um vetor de valores numéricos introduzidos um a um e terminados por um valor vazio**
- O argumento **what** pode ser **usado** para **determinar o tipo de dados a ler** (para **ler** um **vetor de strings**, deve **usar-se “character”**)
- O argumento **file**, **permite** que a **leitura** seja **realizada a partir de um arquivo**

- **Leitura de Dados**

- A **função `read.table`**, permite *ler valores de uma tabela a partir de um arquivo de texto, assumindo que cada linha tem o mesmo número de elementos, separados por um símbolo pré-definido*



A screenshot of a text editor window titled "Tab-Separated-Values.tsv". The window displays a table with three rows and five columns of names, separated by tabs. The names are: Sal, Sue, Bob, Carl, Wanda in the first row; Egbert, Fred, Danita, Shirley, Scott in the second row; and Quincy, Bertha, Jane, Julia, Frank in the third row.

Sal	Sue	Bob	Carl	Wanda
Egbert	Fred	Danita	Shirley	Scott
Quincy	Bertha	Jane	Julia	Frank

- ***Leitura de Dados***

- **Exemplo (01): - Parte 1**

- ***Ler uma tabela*** em que os ***campos estão separados por espaços*** (ou ***tabs***) e a ***primeira linha contém o cabeçalho de cada uma das colunas***

```
tabela = read.table("my_table.txt", header=TRUE)
```

- **Leitura de Dados**

- Exemplo (01): - Parte 2

```
tabela = read.table("my_table.txt", header=TRUE)
```

- O argumento **header** indica se a **1ª linha contém** o nome dos **campos**, neste caso, esse **argumento** tem o **valor TRUE** (ou apenas **T**) o que **indica** que a **1ª linha corresponde** aos **nomes dos campos**, serão **utilizados como nomes de colunas** do **data frame** resultante, sendo **substituído apenas** os **caracteres não aceitos**, com **\$** ou **]**, que **serão substituído** por **.** (ponto)
 - **Caso o arquivo não tenha** um **cabeçalho**, os **nomes das colunas** serão **assumidos** como **"V1", "V2", etc.**

- **Leitura de Dados**

- **Exemplo (01): - Parte 3**

- ```
tabela = read.table("my_table.txt", header=TRUE)
```

- A **função** `read.table` **possui diversos argumentos**, os quais podem ser **consultados** por meio do `help(read.table)`
    - Esses permitem, por exemplo, **alterar o separador** a ser usado (argumento `sep`), **definir a priori o número de linhas a ler** (`nrows`), o **separador decimal** (`dec`), as **strings usadas para representar valores omitidos** (`na.string`)

- *Leitura de Dados*

- Exemplo (01): - Parte 4

- ```
tabela = read.table("my_table.txt", header=TRUE)
```

- *Quando inserir o `file.choose()` como argumento de arquivo, será **apresentado** uma **interface** para **permitir** ao **usuário selecionar** um **arquivo específico** em seu **SO***

- ***Leitura de Dados***

- **Exemplo (02):**

- A *função* **read.csv** *permite ler dados* do *formato* CSV (**C**omma **S**eparated **V**alues)
 - Essa *função invoca* **read.table** com *distintos argumentos*, que por padrão, *define* a *vírgula* como *separador* e o *valor* do *argumento* **header** *configurado* como **TRUE**
 - O *resultado desta função* é um *objeto* do *tipo* **data frame**

```
tabela = read.csv("vendas.csv")
```

- ***Leitura de Dados***

- **Exemplo (03): - Parte 1**

- ***Assumindo*** que o ***arquivo*** de ***interesse*** para ***carregar*** ***esteja disponível*** na ***Internet***, é, na maioria dos casos, ***possível fazer download*** pelos ***métodos habituais***, ***armazenando-o*** numa ***pasta local***
 - ***É possível realizar o download*** dos ***arquivos***, usando a ***função*** **download.file**

- ***Leitura de Dados***

- **Exemplo (03): - Parte 2**

```
> fileUrl = "http://archive.ics.uci.edu/ml/machine-learning-databases/ecoli/ecoli.data"
```

```
> download.file(fileUrl, destfile="ecoli.csv")
```

```
trying URL 'http://archive.ics.uci.edu/ml/machine-learning-databases/ecoli/ecoli.data'  
Content type 'text/plain; charset=UTF-8' length 19488 bytes (19 KB)  
downloaded 19 KB
```

- *Leitura de Dados*

- Exemplo (03): - Parte 3

- *Após o download*, lendo o arquivo via *função* **read.table**

```
> ecoli = read.table("ecoli.csv")
```

```
> dim(ecoli)
```

```
[1] 336  9
```

- ***Leitura de Dados***

- **Exemplo (03): - Parte 4**

- *Apresentando uma **pequena amostra** do **arquivo** [ecoli.csv](#)*

```
> head(ecoli)
```

```
      V1 V2 V3 V4 V5 V6 V7 V8 V9
1 AAT_ECOLI 0.49 0.29 0.48 0.5 0.56 0.24 0.35 cp
2 ACEA_ECOLI 0.07 0.40 0.48 0.5 0.54 0.35 0.44 cp
3 ACEK_ECOLI 0.56 0.40 0.48 0.5 0.49 0.37 0.46 cp
4 ACKA_ECOLI 0.59 0.49 0.48 0.5 0.52 0.45 0.36 cp
5 ADI_ECOLI 0.23 0.32 0.48 0.5 0.55 0.25 0.35 cp
6 ALKH_ECOLI 0.67 0.39 0.48 0.5 0.36 0.38 0.46 cp
```


- ***Leitura de Dados***

- **Observações: - Parte 1**

- A **função readLines** pode ser **utilizada** para **ler um arquivo texto**, retornando um **vetor de strings** que **poderá ser posteriormente manipulado/processado**
 - O R possui ainda diversas funções que permitem ler arquivos em diversos formatos, tais como o formato XLSX (Excel), o JSON e o XML
 - O package XML inclui um conjunto de funções que permite ler arquivos em formatos estruturados como o XML e HTML

- ***Leitura de Dados***

- **Observações: - Parte 2**

- O ***package*** **RJSONIO** ***permite*** ***manipular*** arquivos/documentos no ***formato*** **JSON**



- ***Escrita de Dados***

- **Exemplo (01):**

- ***Existem diversas formas de *exibir valores* no R***
 - ***A função mais simples é a função `print`***

```
> x ← c(2, 3, 5, 9)
```

```
> print(x)
```

```
[1] 2 3 5 9
```

- ***Essa função é chamada sempre que *digitamos o nome de um objeto na linha de comandos****

- ***Escrita de Dados***

- **Exemplo (02):**

- Uma ***opção relevante*** é a ***definição*** do ***número de dígitos numéricos usados*** quando se ***exibe valores numéricos***
 - ***Este valor pode*** ser ***alterado*** com a ***função*** **options**
 - O ***exemplo abaixo configura*** o ***valor*** em **5** (o ***default*** é **7**)

> options(digits=5)

- ***Escrita de Dados***

- **Exemplo (03):**

- A **função** **cat** pode ser **utilizada** com o **mesmo propósito** de **escrita de dados**

```
> cat("Este eh o valor da variavel x: ", x, "\n")
```

```
Este eh o valor da variavel x: 2 3 5 9
```


- ***Escrita de Dados***

- **Exemplo (04): - Parte 1**

- A **função** **cat** **permite ainda enviar o conteúdo** para **um arquivo**
 - A **função cria um arquivo** onde é **armazenado** o que **mandamos exibir; caso o arquivo já exista, é reescrito**

```
> cat(file = "resultado.txt", "Este eh o valor da variavel x: ",  
x, "\n")
```

- ***Escrita de Dados***

- **Exemplo (04): - Parte 2**

- ***Caso desejarmos adicionar algum conteúdo ao arquivo, utilizamos a opção `append`***

`> cat(file = "resultado.txt", "Esta linha eh adicionada depois da linha anterior no mesmo arquivo\n", append = TRUE)`

- ***Escrita de Dados***

- **Exemplo (05): -Parte 1**

- A **função** `write.table` pode ser **utilizada** para **armazenar uma tabela** (`data frame` ou `matriz`), **tanto** no **display** como num **arquivo** (*inverso da função `read.table`*)
 - **Criamos uma matriz** com **duas linhas** e **cinco colunas** e, posteriormente, **utilizamos** a **função** para **imprimi-la** no **display** e no **arquivo**

```
> x ← matrix(1:10, ncol = 5)
```

```
> write.table(x)
```

```
"V1" "V2" "V3" "V4" "V5"
```

```
"1" 1 3 5 7 9
```

```
"2" 2 4 6 8 10
```

- ***Escrita de Dados***

- **Exemplo (05): -Parte 2**

- A **função** `write.table` pode ser **utilizada** para **armazenar uma tabela** (`data frame` ou `matriz`), **tanto** no **display** como num **arquivo** (*inverso da função `read.table`*)
 - **Criamos uma matriz** com **duas linhas** e **cinco colunas** e, posteriormente, **utilizamos** a **função** para **imprimi-la** no **display** e no **arquivo**

```
> write.table(x, file = "tabela.txt")
```

- ***Escrita de Dados***

- **Exemplo (06):**

- ***Uma forma simples de enviar para um arquivo o resultado dos comandos digitados no interpretador consiste na utilização da função `sink`, a qual redireciona o resultado dos comandos para o arquivo***

```
> x ← seq(1, 10, 0.1)
```

```
> sink("saida.txt")
```

```
> x
```

```
> x^3
```

```
> sink()
```


- *Escrita de Dados*

- Exemplo (07):

- *Adicionando* ao *final* do *arquivo* “saida.txt”, o *resultado* do *comando* `cumsum`

```
> sink("saida.txt", append = T)
> cumsum(x)
> sink()
```

- ***Escrita de Dados***

- **Exemplo (08):**

- **Utilização** do **argumento** **split**, o qual **permite imprimir o resultado** dos **comandos** no **display** e no **arquivo**, **simultaneamente**

```
> sink("saida.txt", append = T, split = T)
```

```
> summary(x)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.00	3.25	5.50	5.50	7.75	10.00

```
> sink()
```

- *Escrita de Dados*

- Exemplo (09): - Parte 1

- As *funções* **save** e **load** *permitem armazenar* em *arquivo* os *conteúdos associados* a *objetos* e, *posteriormente, carrega-los*

```
> x ← -50:50
```

```
> f ← function(x) if (x < 0) 0 else x
```

```
> save(x, f, file="dados.Rdata")
```

```
> load("dados.Rdata")
```

```
> f(x)
```

```
[1] 0
```

Warning message:

In if (x < 0) 0 else x :

the condition has length > 1 and only the first element will be used

- ***Escrita de Dados***

- **Exemplo (09): - Parte 2**

```
> x ← -50:50  
> f ← function(x) if (x < 0) 0 else x  
> save(x, f, file="dados.Rdata")  
> load("dados.Rdata")  
> f(x)
```

- A **função** **save** **indica** **quais** **os** **objetos** **que** **serão** **armazenados**
- O **conteúdo** é **armazenado** no **arquivo** em **formato binário** (*apenas interpretado pelo próprio R*)

- ***Escrita de Dados***

- **Observações:**

- *Similar* as ***funções de leitura***, para ***exportação de dados***, o R possui ***diversos packages*** que ***contém funções adequadas*** para ***vários tipos de arquivos***
 - O ***package foreign*** permite ***exportar dados*** para ***aplicações*** como ***SAS*** ou ***Octave***
 - O ***package WriteXLS*** permite a ***escrita*** de ***arquivos*** em ***formato Excel***

- ***Escrita de Dados***

- Observações:

- O **package XML** pode ser **usado** para **escrever arquivos XML**
 - O **package prettyR** inclui **funções úteis** para **escrita de resultados** em **formato HTML**



Exercícios

